Performance Comparison of CPU and CUDA Matrix Multiplication:

A Study of Naive vs. Tiled Shared-Memory Implementations

Author: Ebrahim khodadadi
Course: Operating Systems
Date: October 2026

Abstract

This report presents a comparative performance analysis of three matrix multiplication implementations for square matrices of size 1024×1024: (1) a single-core CPU implementation, (2) a naive CUDA kernel, and (3) an optimized tiled CUDA kernel that utilizes shared memory. Experiments were conducted on an NVIDIA GeForce RTX 3050 Laptop GPU. The results show that the naive CUDA implementation achieves approximately **670×** speedup over the single-core CPU version, while the tiled shared-memory version further improves performance by about **31%**, reaching roughly **880×** speedup relative to the CPU baseline. The study highlights the critical role of the GPU memory hierarchy and thread cooperation in reducing global memory traffic.

Keywords: CUDA, Matrix Multiplication, Shared Memory, Tiling, Parallel Computing, Performance Optimization

---

1. Introduction

Matrix multiplication is a fundamental operation in scientific computing, machine learning, and computer graphics. Its computational complexity of $O(N^3)$makes it an ideal candidate for studying parallel computing techniques and memory hierarchy optimizations.

Modern GPUs contain thousands of lightweight cores designed for massive data-parallel workloads. However, achieving high performance on GPUs requires careful management of the memory hierarchy, particularly the use of fast on-chip shared memory to reduce costly accesses to global memory.

In this work, we implement and evaluate three versions of matrix multiplication:

- Single-core CPU (baseline)
- Naive CUDA (one thread per output element)
- Tiled CUDA with shared memory

The goal is to quantify the performance gains obtained by moving from sequential CPU execution to massively parallel GPU execution, and further by applying the classic tiling optimization.

---

2. Background

2.1 GPU Architecture Overview

A modern NVIDIA GPU consists of multiple Streaming Multiprocessors (SMs). Each SM contains many CUDA cores, a register file, and a small but extremely fast shared memory region. Threads are grouped into blocks, and blocks are scheduled onto SMs.

Key programming abstractions in CUDA:

- **Thread**: The basic unit of execution
- **Block**: A group of threads that can cooperate via shared memory and synchronization
- **Grid**: A collection of blocks

2.2 Memory Hierarchy

Memory Type	Scope	Latency	Bandwidth	Size
Registers	Per thread	~1 cycle	Very high	Limited per thread
Shared Memory	Per block	~20–30 cycles	~1–10 TB/s	48–100 KB per SM
Global Memory	Entire grid	400–800 cycles	~200–400 GB/s	Several GB

The central optimization principle in CUDA is to maximize data reuse in shared memory and registers while minimizing accesses to global memory.

2.3 Matrix Multiplication

Given matrices $A, B \in \mathbb{R}^{N \times N}$, the product $C = A \times B$is defined as:

$$C_{ij} = \sum_{k=0}^{N-1} A_{ik} \cdot B_{kj}$$

The total number of floating-point operations is approximately $2N^3$.

---

3. Implementation

3.1 CPU Single-Core Implementation

A straightforward triple-loop implementation was used. Each element of the output matrix is computed sequentially on a single CPU core. Timing was performed using std::chrono::high_resolution_clock with one warm-up run and five measurement runs.

3.2 Naive CUDA Kernel

In the naive implementation, a 2D grid of threads is launched where each thread is responsible for computing exactly one element of matrix C. The kernel directly reads from global memory in the inner loop:

sum += A[row * N + k] * B[k * N + col];

This approach leads to massive redundant global memory traffic, as each element of $A$is read Ntimes and each element of $B$is read Ntimes.

3.3 Tiled CUDA Kernel with Shared Memory

The tiled implementation divides the computation into tiles of size 16×16. Threads within a block cooperatively load tiles of $A$and Binto shared memory, synchronize using __syncthreads(), perform the partial dot products from shared memory, and then proceed to the next tile.

This strategy significantly reduces global memory accesses by exploiting data reuse within each tile.

---

4. Experimental Setup

- **Matrix size**: 1024×1024(single precision)
- **GPU**: NVIDIA GeForce RTX 3050 Laptop
- **Block size**: 16×16threads
- **Measurement method**:
    - CPU: std::chrono (1 warm-up + 5 runs)
    - GPU: CUDA Events (2 warm-up + 5 runs)
- **Performance metric**: GFLOPS, calculated as $\frac{2N^3}{t \times 10^9}$

Only kernel execution time was measured for the GPU versions (data transfer time over PCIe was excluded to focus on computational performance).

---

5. Results and Discussion

Implementation	**Avg. Time (ms)**	**Best Time (ms)**	**Avg. GFLOPS**	**Best GFLOPS**	**Speedup vs CPU**
CPU Single-core	3295.25	2938.47	0.65	0.73	1×
CUDA Naive	4.91	4.91	437.17	437.36	~670×
CUDA Tiled (Shared Memory)	**3.74**	**3.73**	**573.91**	**575.82**	**~880×**

Analysis

1. **CPU → Naive CUDA**:
   The enormous speedup (approximately 670×) comes from massive parallelism. The RTX 3050 Laptop can execute tens of thousands of threads concurrently, whereas the CPU version uses only a single core.

2. **Naive → Tiled**:
   The tiled version achieves an additional **31%** performance improvement. By loading data into shared memory once per tile and reusing it multiple times, the kernel significantly reduces pressure on the global memory subsystem. This makes the computation less memory-bound and more compute-bound.

3. **Absolute Performance**:
   The best tiled performance of ~576 GFLOPS is still well below the theoretical peak of the RTX 3050 Laptop (several TFLOPS). This is expected, as further optimizations (larger tiles, vectorized loads, better occupancy tuning, double buffering, etc.) are possible but beyond the scope of this study.

6. Conclusion

This study demonstrated the dramatic performance benefits of GPU computing for matrix multiplication. Moving from a single-core CPU implementation to a naive CUDA kernel yielded a speedup of approximately 670×. Applying the classic tiling optimization with shared memory provided an additional 31% improvement, resulting in an overall speedup of roughly 880× compared to the CPU baseline.

The results underscore two fundamental principles of high-performance GPU programming:

1. Massive parallelism is highly effective for data-parallel problems.
2. Careful management of the memory hierarchy—particularly the use of shared memory—is essential to approach the computational peak of the hardware.

Future work could explore larger matrix sizes, different tile sizes, and more advanced optimization techniques such as cooperative groups or tensor cores.

References

- NVIDIA CUDA C++ Programming Guide
- Kirk, D. B., & Hwu, W. W. (2016). *Programming Massively Parallel Processors*
- Volkov, V. (2010). Better performance at lower occupancy

